

Fall 2026 - Assignment 1

Instructor: Dr. Behrouz Far

MAS Requirements and Design

Topic: Multi-Agent Approach for Automatic Software Refactoring

Submitted by: Mohammad Kaosain Akbar

SENG 696 - Agent-Based Software Engineering

Table of Contents

1	Summary		
1.1		<i>System Background and Goal</i>	
1.2		<i>Requirement</i>	
2	Design Document (MaSE)		
2.1		<i>Capturing Goals</i>	
2.1.1			Goal Hierarchy
2.2		<i>Applying Use Case</i>	
2.2.1			Use Case
2.2.2			Sequence Diagram
2.3		<i>Refining Roles</i>	
2.3.1			Role Diagram
2.3.2			Role Diagram and Tasks
2.4		<i>Creating Agent Classes</i>	
2.4.1			Agent Description and Agent Class Diagram
2.5		<i>Agent Internal Architecture</i>	
2.5.1			Conversation
2.5.2			Agent Architecture
2.5.3			Deployment Diagram

List of Tables and Figures

Table 1: Use Case Description for Automatic Java Class Refactoring

Table 2: The Process Steps for the Agents

Table 3: Agent Roles, Tasks and External Resources

Table 4: Agent Communication Messages and Events

Table 5: Agent Tool and Internal State

Figure 1. Goal hierarchy for the automatic software refactoring system

Figure 2. Primary use-case sequence for the automatic refactoring workflow

Figure 3. Role diagram and primary inter-role communication

Figure 4. Roles, internal tasks, and external tools.

Figure 5. Preliminary agent class diagram.

Figure 6. Conversation for collecting context and producing the refactoring plan

Figure 7. Conversation between the Generator and Compiler during build validation

Figure 8. Conversation between the Generator and Tester during behavioral validation

Figure 9. Generic internal architecture used by the specialized LLM agents

Figure 10. Preliminary deployment diagram for the automatic software refactoring system

1. Summary

1.1. System Background and Goal

Software refactoring improves the internal structure and design of software without changing its observable behavior. In large codebases, manual refactoring can be time-consuming and error-prone. This project will present a multi-agent approach that automates the refactoring workflow by distributing planning, code generation, compilation validation, and behavioral testing across specialized agents.

The system will tentatively operate on a Java project and will process a target class using around four collaborating agents: a Context-Aware Planner Agent, a Refactoring Generator Agent, a Compiler Agent, and a Tester Agent. The agents will cooperate through structured messages and tool calls so that a proposed refactoring will not be accepted until it compiles successfully and its tests pass.

The Planner Agent will gather context using the project source code, a jdeps dependency graph, and class/method quality metrics from DesigniteJava. It then will produce a structured refactoring plan. The Generator Agent will apply that plan to create a modified Java class. The Compiler Agent will validate the updated project and when compilation fails, returns a summarized error and fixing instruction to the Generator. After successful compilation, the Tester Agent will locate existing developer tests, generates additional regression tests with EvoSuite when needed, and executes the test suite using Maven.

This Report 1 focuses on the multi-agent requirements and preliminary MaSE design of the proposed system. The diagrams in this report are project-specific MaSE design artifacts that describe the goals, roles, interactions, agent classes, conversations, internal architecture, and deployment of the system.

1.2. Requirements

The baseline functional requirements of the system are:

- R1. The system shall accept a Java project and a target Java class, or iterate through project classes when running the complete workflow.
- R2. The Context-Aware Planner Agent shall obtain the target class source code and the source code of relevant first-degree dependent classes.
- R3. The Planner Agent shall use jdeps to obtain class-level dependency information and DesigniteJava 2.5.2 to obtain class and method quality metrics.
- R4. The Planner Agent shall identify refactoring opportunities and produce a structured refactoring plan containing class-level and/or method-level instructions.
- R5. The Refactoring Generator Agent shall apply the plan to the target class while retaining the contextual information supplied by the Planner.
- R6. The Compiler Agent shall compile the modified project using Maven. When compilation fails, it shall analyze the compiler logs and send a corrective instruction to the Generator.
- R7. Compilation correction shall continue until compilation succeeds or the maximum feedback-loop limit is reached. The current design uses a maximum of 20 iterations.
- R8. After successful compilation, the Tester Agent shall locate relevant developer-written tests and generate additional regression tests using EvoSuite when required.

- R9. The Tester Agent shall execute the developer and generated tests using Maven. When a test fails, it shall summarize the failure and request a correction from the Generator.
- R10. Testing correction shall continue until the tests pass or the maximum feedback-loop limit is reached. If validation does not succeed within the limit, the refactoring shall not be retained and the original class shall be kept.
- R11. A successful refactoring shall be retained only when both compilation and behavioral validation succeed.
- R12. The system shall preserve clear separation of responsibilities among the four specialized agents and expose external analysis/build/test tools through tool interfaces rather than modeling the tools themselves as autonomous agents.

Key quality requirements are:

- Q1. Behavior preservation: accepted refactorings must preserve the behavior exercised by the available developer and generated regression tests.
- Q2. Build correctness: accepted changes must compile successfully.
- Q3. Context awareness: refactoring decisions should use source dependencies and quality metrics rather than only the isolated target class.
- Q4. Modularity: planning, code generation, compilation validation, and testing validation should remain separate agent responsibilities.
- Q5. Autonomy: once a project/class is provided, agents should execute the workflow and corrective loops without manual intervention except for stopping or configuration.

2. Design Document (MaSE)

The following preliminary design applies the MaSE-style stages used in the course sample: capturing goals, applying use cases, refining roles, creating agent classes, modeling conversations, defining agent architecture, and describing deployment. The design organizes the system around four specialized agents with clearly separated responsibilities.

2.1. Capturing Goals

2.1.1. Goal Hierarchy

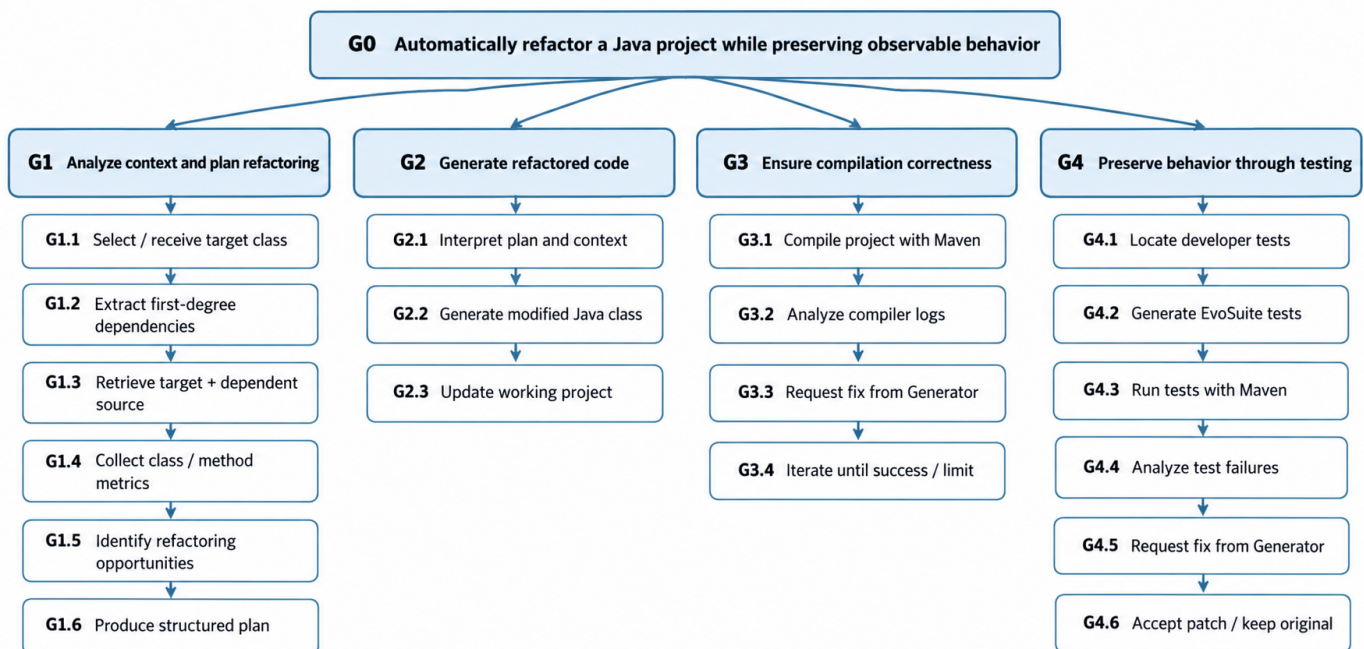


Figure 1. Goal hierarchy for the automatic software refactoring system.

Goal Mapping:

- Context-Aware Planner Role: G1, G1.1, G1.2, G1.3, G1.4, G1.5, G1.6
- Refactoring Generator Role: G2, G2.1, G2.2, G2.3
- Compiler Role: G3, G3.1, G3.2, G3.3, G3.4
- Tester Role: G4, G4.1, G4.2, G4.3, G4.4, G4.5, G4.6

2.2. Applying Use Cases

2.2.1. Use Case

Table 1: Use Case Description for Automatic Java Class Refactoring

Brief Description:	Automatically analyze and refactor a Java class, then validate the change through compilation and testing.
Precondition(s):	The Java/Maven project is available. The selected target class is accessible. The original project is in a known working state for the validation activities being performed.
Post Condition(s):	If compilation and tests pass, the refactored class is retained. If validation cannot be completed successfully within the configured iteration limit, the original class is retained and the workflow proceeds without applying that failed refactoring.

Table 2: The process steps for the agent

Process Step	Description
1	The user or project runner provides the Java/Maven project and target class to the system.
2	The Planner Agent invokes jdeps to create or query the class dependency graph and identify first-degree dependent classes.
3	The Planner Agent invokes DesigniteJava to collect class and method quality metrics for the target class.
4	The Planner Agent uses the code-search module to retrieve the target class and relevant dependent source code.
5	The Planner Agent combines source context, dependency information, and quality metrics to identify refactoring opportunities.
6	The Planner Agent produces a structured refactoring plan and sends it to the Refactoring Generator Agent.
7	The Generator Agent applies the plan and writes a refactored version of the target class to the working project.
8	The Generator Agent notifies the Compiler Agent that the updated code is ready.
9	The Compiler Agent invokes Maven compilation and receives the compilation status and logs.
10	If compilation fails, the Compiler Agent analyzes the logs, creates an error summary and fixing instruction, and sends them to the Generator Agent.

11	The Generator Agent revises the class and resubmits it for compilation. The correction loop can repeat up to 20 iterations in the original design.
12	When compilation succeeds, the Compiler Agent transfers control to the Tester Agent.
13	The Tester Agent uses the dependency information/code search to locate relevant developer-written tests and invokes EvoSuite to generate additional regression tests from the original program when required.
14	The Tester Agent invokes Maven to execute the relevant developer and generated tests.
15	If a test fails, the Tester Agent analyzes the failure, generates a summary and fixing instruction, and sends them to the Generator Agent.
16	The Generator Agent revises the class and resubmits it for testing; the testing feedback loop can repeat up to 20 iterations.
17	If all required tests pass, the system keeps the refactored code. If validation still fails after the iteration limit, the system keeps the original class and continues without applying the failed refactoring.

2.2.2. Sequence Diagram

The primary sequence diagram models the complete successful path together with the two corrective feedback loops. External utilities are shown as tools rather than agents because their behavior is invoked and controlled by the specialized LLM agents.

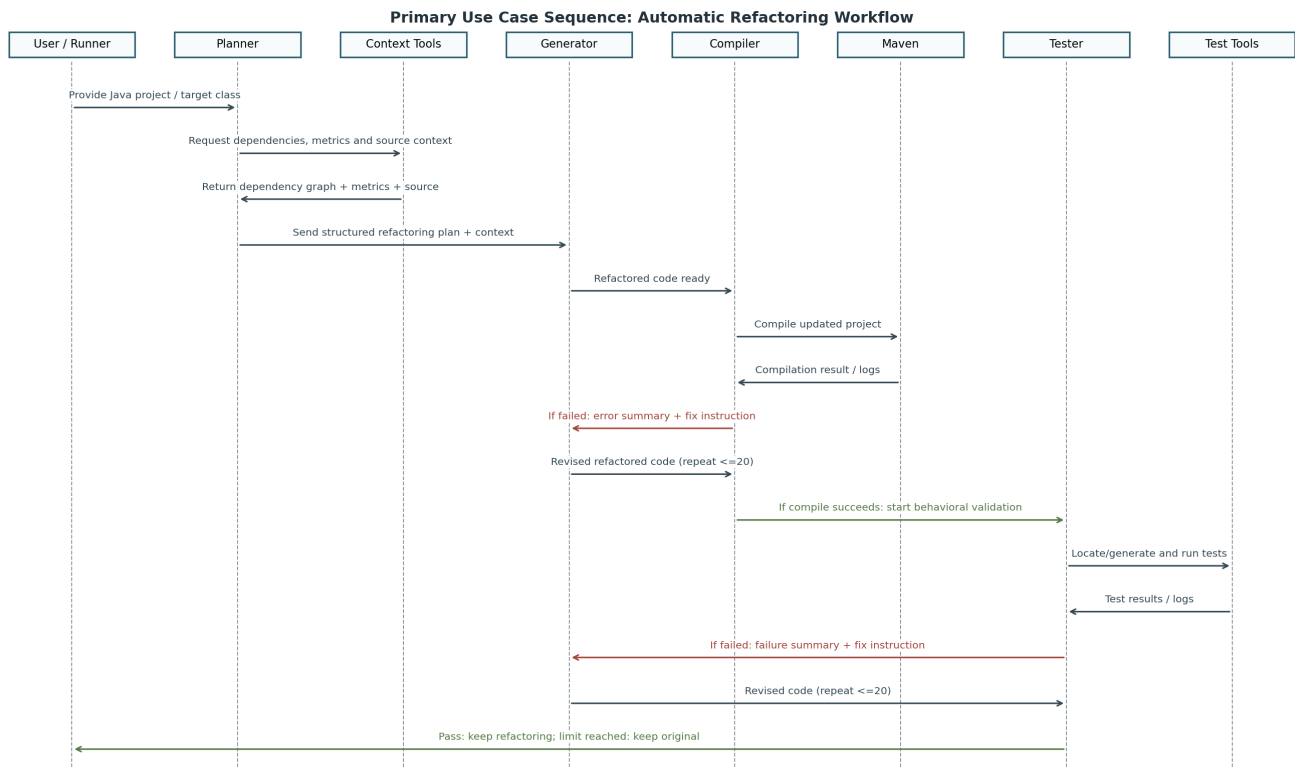


Figure 2. Primary use-case sequence for the automatic refactoring workflow.

2.3. Refining Roles

2.3.1. Role Diagram

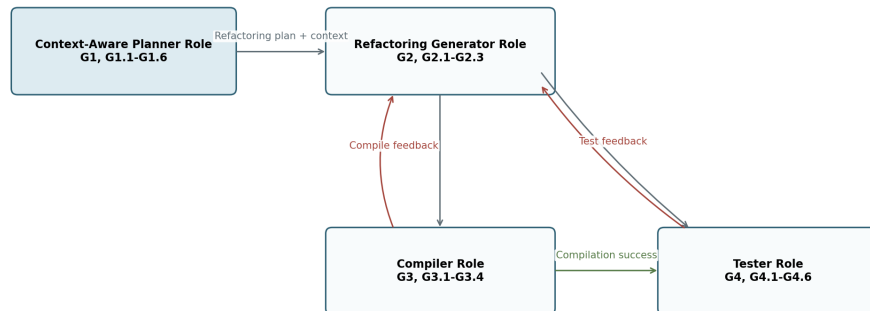


Figure 3. Role diagram and primary inter-role communication.

The role model follows the sequential refactoring workflow. The Planner owns context analysis and planning, the Generator owns code transformation, the Compiler owns build validation and compilation feedback, and the Tester owns behavioral validation and test feedback. The Generator is the common correction point because both validation roles may request revisions.

2.3.2. Role Diagram & Tasks

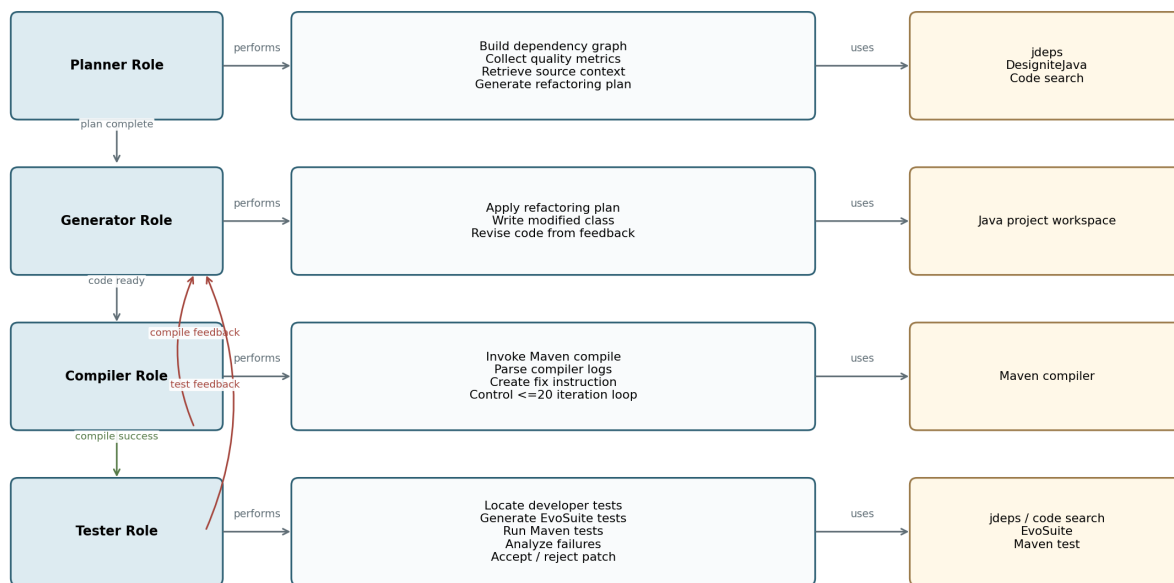


Figure 4. Roles, internal tasks, and external tools.

Table 3: Agent Roles, Tasks and External Resources

Role	Main Tasks	External Resources / Outputs
Context-Aware Planner	Build dependency context; obtain quality metrics; retrieve source; identify opportunities; create structured plan.	jdeps, DesigniteJava, code search; output: RefactoringPlan.
Refactoring Generator	Interpret plan; generate modified Java code; revise code when compiler or tester returns feedback.	Project workspace; output: updated target class.
Compiler	Invoke Maven compile; inspect logs; summarize failures; manage compilation correction loop.	Maven compiler; output: CompileSuccess or CompileFixRequest.
Tester	Locate developer tests; generate EvoSuite regression tests; run tests; inspect failures; decide whether patch can be retained.	jdeps/code search, EvoSuite, Maven test; output: TestPass or TestFixRequest.

2.4. Creating Agent Classes

2.4.1. Agent Description & Agent Class Diagram

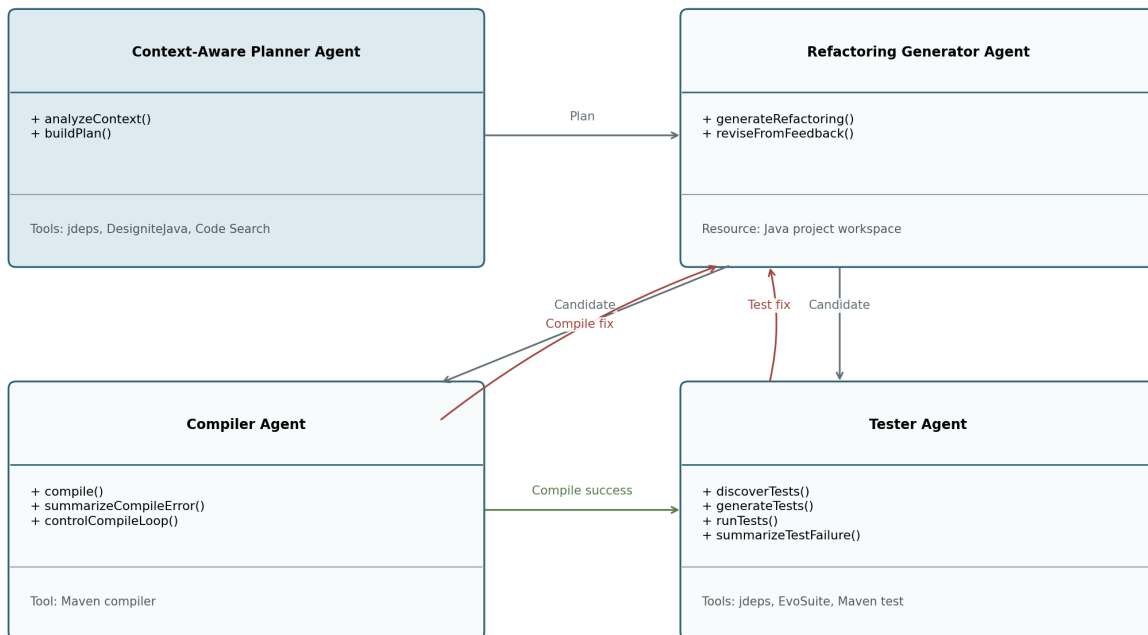


Figure 5. Preliminary agent class diagram.

Context-Aware Planner Agent

- Takes a Java class as the starting point for analysis.
- Calls jdeps to obtain class-level dependencies and uses the code-search module to retrieve the target class and first-degree dependent classes.
- Calls DesigniteJava 2.5.2 to obtain quality metrics used as context for planning. The planning stage uses the metrics themselves rather than DesigniteJava's labeled smell detections [1].
- Uses the LLM reasoning core to identify refactoring opportunities and produces a structured plan that can include class-level and method-level instructions.
- Sends the plan and required source context to the Refactoring Generator Agent.

Refactoring Generator Agent

- Receives the original target source, dependent-class context, quality information, and the structured refactoring plan.
- Generates a refactored Java class while treating the Planner's instructions as the primary objective.
- Updates the working project and triggers the validation workflow.
- Receives compilation-error or test-failure feedback and revises the code while continuing to follow the original plan.

Compiler Agent

- Invokes the Maven compiler on the modified project without running tests.
- Receives compilation status and compiler logs.
- If compilation fails, analyzes the logs, summarizes the error, creates a fixing instruction, and sends the feedback to the Generator.
- Controls a self-reflection/correction loop of up to 20 iterations [1].
- When compilation succeeds, transfers the workflow to the Tester Agent.

Tester Agent

- Ensures that refactoring does not alter observable behavior.
- Uses the dependency graph/code-search capability to identify relevant developer-written tests.
- Invokes EvoSuite to generate regression tests from the original program when additional tests are needed.
- Runs developer-written and generated tests using Maven, analyzes failures, and sends a fixing request to the Generator when necessary.
- Controls a test-correction loop of up to 20 iterations; if the patch still fails, the original class is retained instead of the refactoring.

2.5. Agent Internal Architecture

2.5.1. Conversations

The main agent conversations are modeled as message exchanges around three stages: planning, compilation validation, and testing validation. The message names below are preliminary protocol labels for the course implementation.

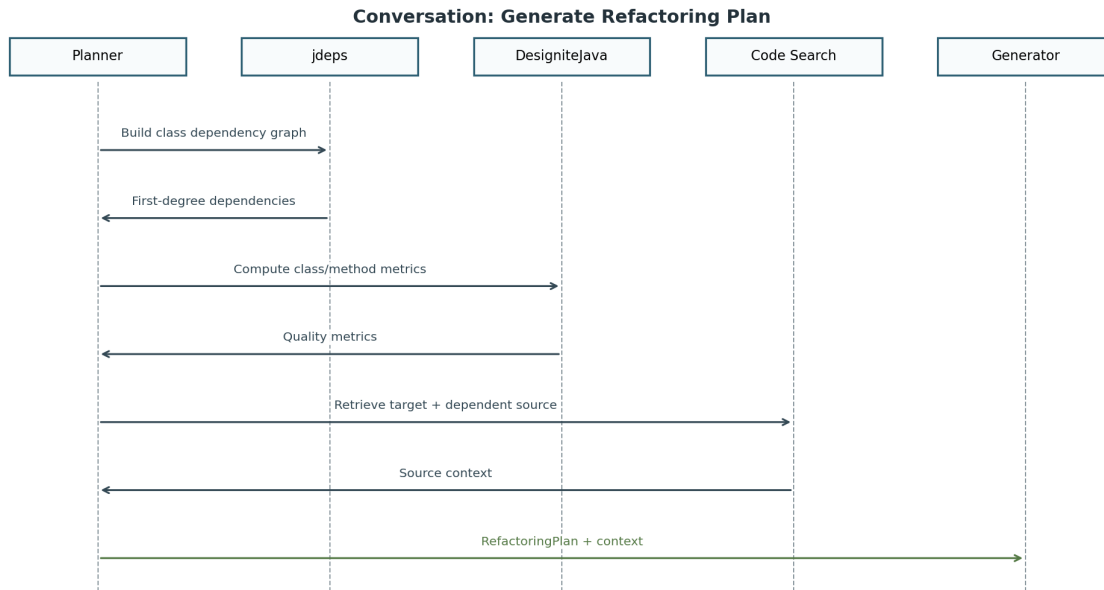


Figure 6. Conversation for collecting context and producing the refactoring plan.

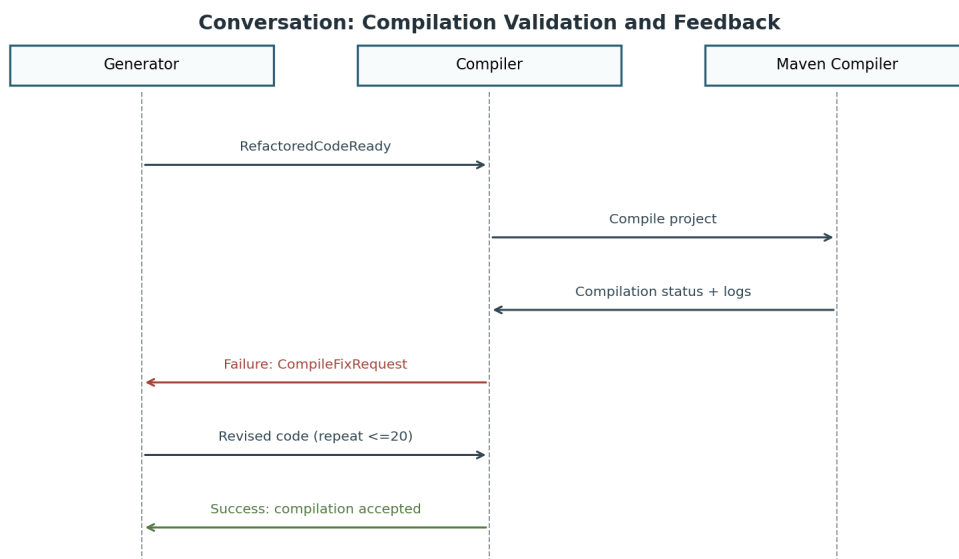


Figure 7. Conversation between the Generator and Compiler during build validation.

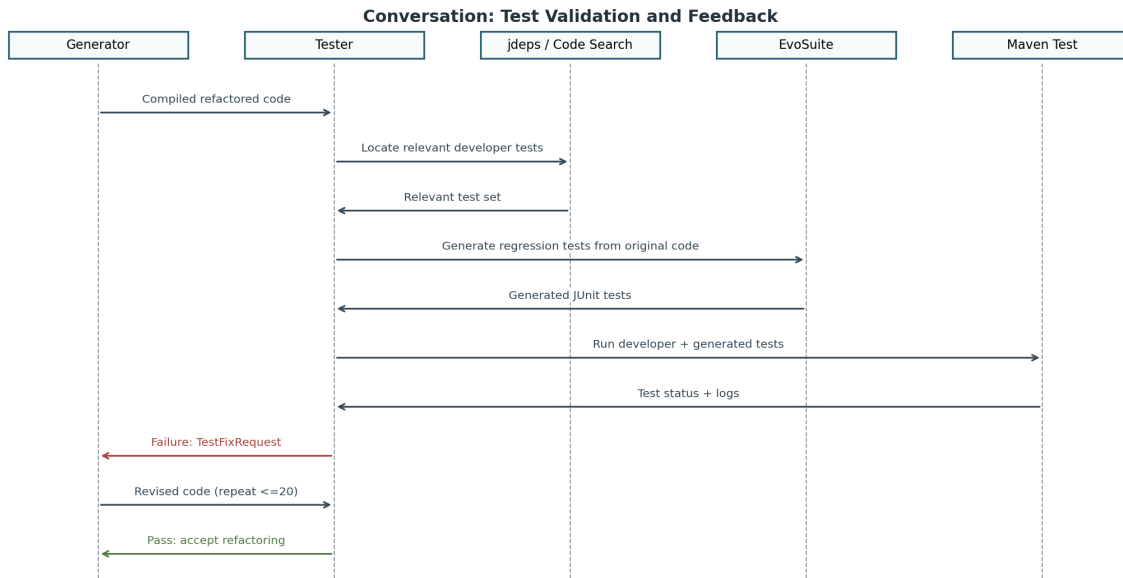


Figure 8. Conversation between the Generator and Tester during behavioral validation.

Table 4: Agent Communication Messages and Events

Message / Event	Sender -> Receiver	Content / Purpose
RefactoringPlan	Planner -> Generator	Structured class/method refactoring instructions plus contextual source information.
RefactoredCodeReady	Generator -> Compiler	Signals that the working project contains a new candidate refactoring.
CompileFixRequest	Compiler -> Generator	Compilation error summary and concrete correction instruction.
CompileSuccess	Compiler -> Tester	Signals that syntax/build validation succeeded and testing may begin.
TestFixRequest	Tester -> Generator	Failing-test information, summarized cause, and correction instruction.
TestPass / Reject	Tester -> workflow	Accept the candidate when tests pass; otherwise reject/rollback when the iteration limit is exhausted.

2.5.2. Agent Architecture

The four functional agents and their external tool interactions define the high-level workflow. The following internal architecture is a preliminary MaSE design decision for this project and provides a consistent implementation structure for all four agent types.

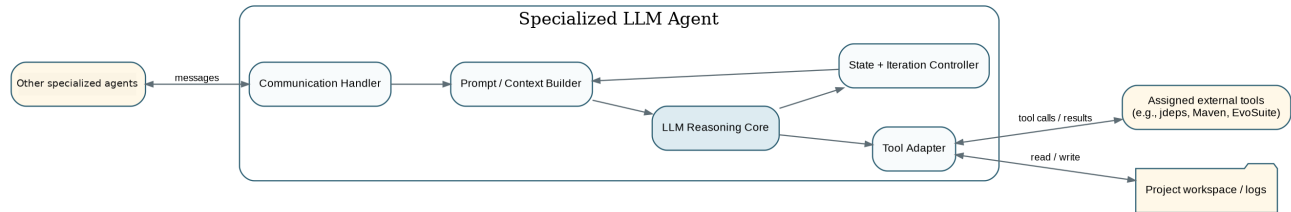


Figure 9. Generic internal architecture used by the specialized LLM agents.

Internal components:

- **Communication Handler:** receives messages from other agents and sends plans, statuses, or correction requests.
- **Prompt / Context Builder:** converts the role-specific task, project context, tool results, and previous feedback into the LLM input required by that agent.
- **LLM Reasoning Core:** performs planning, code generation, or error/failure analysis according to the role.
- **Tool Adapter:** exposes only the tools assigned to the role, such as jdeps, DesigniteJava, Maven, EvoSuite, code search, or project-file access.
- **State + Iteration Controller:** records the current target class and iteration count and enforces the maximum feedback-loop limit for compilation or testing.

Table 5: Agent Tool and Internal State

Agent	Primary Tool Adapter(s)	State of Interest
Planner	jdeps, DesigniteJava, code search	Target class, dependencies, metrics, retrieved source, generated plan
Generator	Project source read/write	Original class, refactoring plan, current candidate, latest compiler/test feedback
Compiler	Maven compiler	Compilation status, error logs, iteration count
Tester	jdeps/code search, EvoSuite, Maven test	Relevant tests, generated tests, failure logs, iteration count, final validation status

2.5.3. Deployment Diagram

The deployment model separates the multi-agent orchestration layer from the Java project workspace and the external tools. The LLM backend may be a cloud API or a locally hosted model, keeping the architecture independent of a single provider.

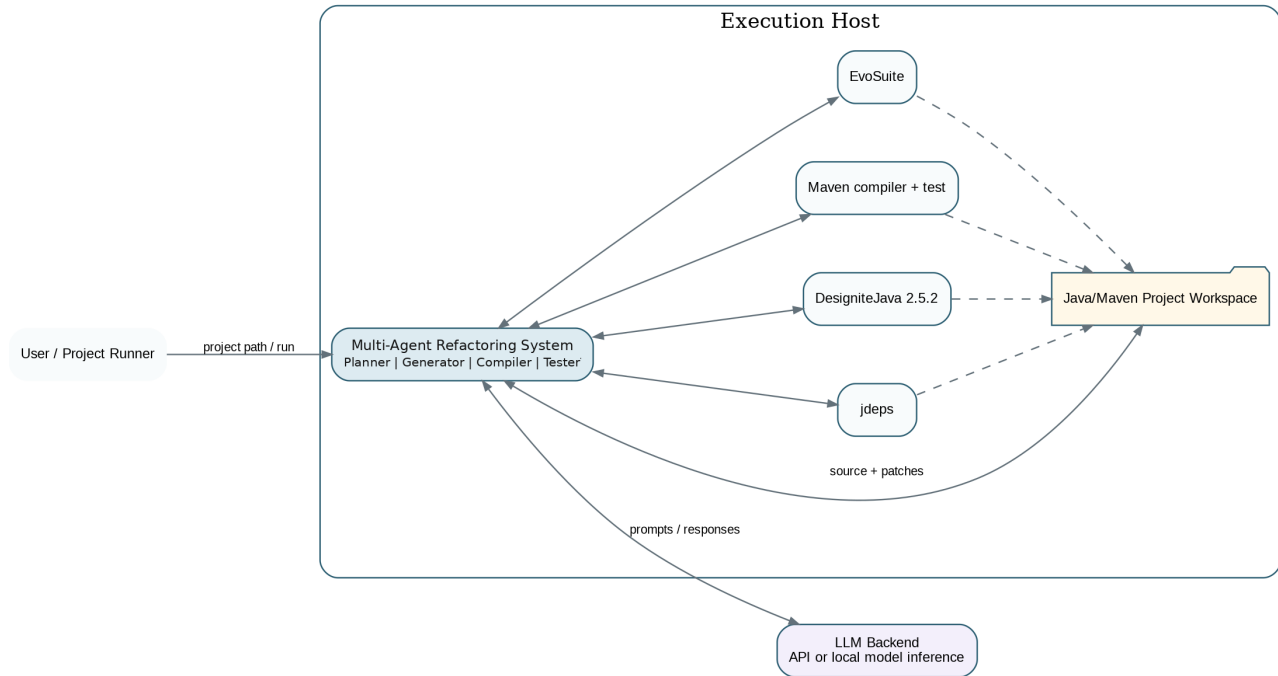


Figure 10. Preliminary deployment diagram for the automatic software refactoring system.

Deployment responsibilities:

- The execution host runs the four-agent workflow and the local Java/Maven analysis and validation tools.
- The Java/Maven project workspace stores the original and candidate source code, compiler logs, test logs, and generated test artifacts needed by the workflow.
- The agent system communicates with an LLM backend for role-specific reasoning and generation.
- jdeps and DesigniteJava provide context to the Planner; Maven provides compilation and test execution; EvoSuite supplies generated regression tests to the Tester.
- The project runner starts the workflow and receives the final accepted/unchanged project result.